AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

NAME: RAAGHAV VEL P

CSE-B

CH.SC.U4CSE24155

Merge Sort:

Code:

```
#include <stdio.h>

int a[100], n;

void merge(int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;
    int temp[100];

    while (i <= mid && j <= high) {
        if (a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= high)
        temp[k++] = a[j++];

    for (i = low; i <= high; i++)
        a[i] = temp[i];
}

void mergeSort(int low, int high) {
    int mid;
    if (low < high) {
        mid = (low + high) / 2;
        mergeSort(low, mid);
        mergeSort(mid + 1, high);
        merge(low, mid, high);
    }
}
```

```c
void main() {
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}
```

Time Complexity: O(nlogn)

Justification:

Merge Sort works using the divide and conquer approach. The given array of size n is repeatedly divided into two halves until each array contains only one element. division process takes log n level At every level of division, the merge operation combines all elements back together. Merging two sorted parts takes O(n) time

n X log n = n log n

Quick Sort:

Code:

```
#include <stdio.h>

int a[100], n;

int partition(int low, int high) {
    int pivot = a[low];
    int i = low + 1;
    int j = high;
    int temp;

    while (i <= j) {
        while (a[i] <= pivot && i <= high)
            i++;
        while (a[j] > pivot)
            j--;

        if (i < j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j;
}

void quickSort(int low, int high) {
    int pos;
    if (low < high) {
        pos = partition(low, high);
        quickSort(low, pos - 1);
        quickSort(pos + 1, high);
    }
}
```

```c
void main() {
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}
```

Time Complexity: O(nlogn) (worst case : n^2)

Justification:

Quick Sort divides the array based on a pivot element.If the pivot divides the array into two nearly equal halves, there are log n levels of recursion if the worst can then n levels and partitioning takes O(n) time

nXlogn=nlogn

and

nXn=n^2